

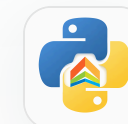


SCIPY 2026

MULTI-MODEL DATA IN PYTHON

ArcadeDB in Python: An In-Process Multi-Model Database

Taewoon Kim | HumemAI | taewoon.kim



`pip install arcadedb-embedded`

THE PROBLEM

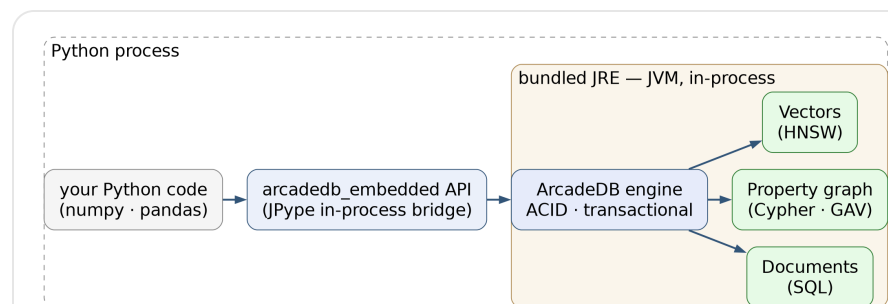
Scientific Python increasingly mixes **three** kinds of data access over the *same* data — records (SQL), relationships (graph), similarity over embeddings (vectors). Today that means **three separate systems + custom ETL + hand-kept consistency**.



THE IDEA · THE CONTRIBUTION

One embedded engine, in-process, one pip install

Documents, a property graph, and HNSW vectors in **one process, one file-backed database, one transaction** — no server. Engine = **ArcadeDB** (Apache-2.0 Java, credited); our contribution is the **Python binding**: JPytype in-process JVM + NumPy/pandas API + a **bundled per-platform JRE** → **20 wheels, no Java to install**.

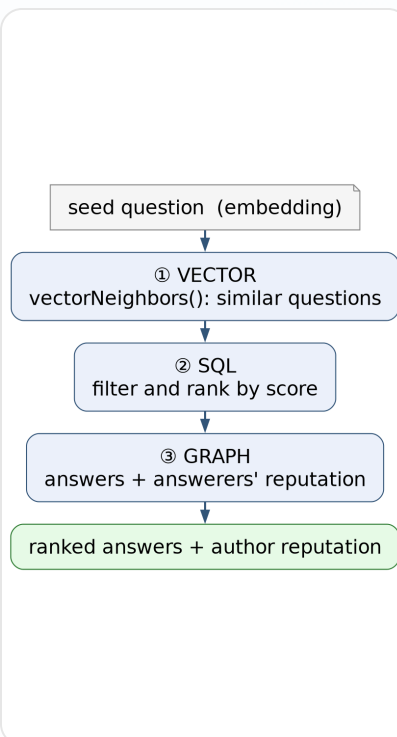


A Java engine sealed inside a Python wheel: one process hosts an in-process JVM (via JPytype) running ArcadeDB over a local file-backed DB; documents/SQL, property graph (Cypher) and HNSW vectors reached through **one handle**. Wheel ≈62 MB, plain `pip install`.

★ THE PAYOFF — ONE WORKFLOW, THREE MODELS

Hybrid retrieval: vector → SQL → graph, one transaction

Given a seed question, find semantically similar ones, keep the well-scored, return their best answers with each author's reputation. **Identifiers flow directly between steps — no serialization, no ETL, no second system.**



```
# 1) VECTOR — questions similar to a seed
cands = db.query("sql",
    "... expand(vectorNeighbors(?,?,?,?)) ...",
    "Question[embedding]", seed, 200, 100).to_list()

# 2) SQL — keep well-scored (ids from step 1)
filt = db.query("sql",
    f"... WHERE id IN {ids} AND score >= 1 ...").to_list()

# 3) CYPHER — traverse to answers + reputation
hits = db.query("cypher",
    f"MATCH (q:Question)-[:HAS_ANSWER]->(a:Answer)
    -[:AUTHORED_BY]->(u:Userx) WHERE q.id IN {fids} ...").to_list()
```

≈16 ms

warm end-to-end (median of 20 reps) over all **213,761 questions** + 208,986 answers.
1 process · 1 transaction · 0 ETL (vector 10 · SQL 5 · Cypher 1.4 ms; native SQL MATCH: 3 ms)

HONEST RESULTS — VS SPECIALIST EMBEDDED PEERS

Fast where it's designed to be

p99 wins **Read latency**: graph point/hop 0.45/0.56 ms beat the graph specialist; tabular 0.17 ms beats DuckDB.

≈7.5× **Graph OLTP** vs LadybugDB at default contracts (≈3,900 vs ≈525 ops/s); parity at matched full durability — disclosed.

0.95 **Vector recall@10** at matched HNSW (Chroma 0.97) — competitive *while* multi-model, with **lower peak memory** (16.8 vs 24.6 GiB, disk-backed index).

...AND WHERE IT ISN'T (WE DON'T HIDE IT)

Specialists win **heavy analytics**: DuckDB ≈9 ms vs ≈1,300 ms (tabular OLAP); LadybugDB ≈66 ms vs ≈800 ms (graph, a **GAV** narrows the gap). WAL-mode **SQLite owns raw point-op throughput** (≈87k vs ≈5.8k ops/s — in-process C is untouchable; durability contracts disclosed per engine). The standing cost is **memory** — a JVM baseline (≈852 MiB tabular) above the lean C peers; startup is sub-second, with an occasional GC tail. But these gaps are a **moving target**: ArcadeDB ships ~monthly and the binding packages each release, so Python benefits automatically.

CODE github.com/humemai/arcadedb-embedded-python

DOCS docs.humem.ai/arcadedb

Apache-2.0 HNSW vector search 10–3.14

matched-param benchmark

pinned versions + run manifest

datasets open

Reproducible and Cross Validated (stats.stackexchange.com) corpus, CC BY-SA; ArcadeDB 26.8.1.dev2 (workflow: do73), DuckDB 1.5.4, SQLite 3.46.1, LadybugDB 0.18.1, Chroma 1.5.9; 8-core cap; median over 5 reps. GenAI assisted language editing; all content reviewed & verified by the author.